tblaase / Net_Practice Public

Code Issues Pull requests

main 1 Branch 0 Tags Go to file Go to file Code

tblaase 3 years ago

my_solutions	changed solution of Level 7	4 years ago
readme_additions	moved subject.pdf to private repo	4 years ago
LICENSE	Initial commit	4 years ago
README.md	Typo and grammer fixes	3 years ago

Net_Practice

My solution and basic explanation on Net_Practice and networking basics.

First of all, thanks to Robin | radelwar for explaining the more in depth stuff to me.
This may not be perfect since I just learned about all of the following myself.
So please, if there is anything wrong or missing, please notify me by reaching out to me (my profile) or create an issue, thanks.

I only created this tutorial, since the project has a few flaws and you have the ability to solve it in a way that's wrong, but the project is still telling you that everything is correct.
And all other tutorials I found used these flaws to simplify the solution.

Contents

Basics

For this project we only use IPv4, so I won't talk about IPv6.
An IPv4-adress is a 32-bit number divided into 4 "blocks", each 8 bits.
i.e.:
`192.168.100.1` turns into `11000000.10101000.01100100.00000001`
So the min. value of one "block" is `0` and the max. value is `255`.
The same logic applies to the network-mask:
`255.255.255.0` turns into `11111111.11111111.11111111.00000000`
Special to the mask is, after one bit was `0` there can't be any `1` bit's anymore.
So the only available numbers are:

- `255 (binary: 11111111)`
- `254 (binary: 11111110)`
- `252 (binary: 11111100)`
- `248 (binary: 11111000)`
- `240 (binary: 11110000)`
- `224 (binary: 11100000)`
- `192 (binary: 11000000)`
- `128 (binary: 10000000)`
- `0 (binary: 00000000)`

Through which `255.255.255.0` is a valid mask
and `255.255.128.128` is **not** a valid mask.

In order to have the ability to send packages between two IP-addresses they either need to be part of the same network or they need to be connected by a router which is part of both subnets.

back to contents

Special IP-ranges

The following special address-ranges are reserved for Private Networks:
`10.0.0.0 – 10.255.255.255`
`172.16.0.0 – 172.31.255.255`
`192.168.0.0 – 192.168.255.255`

The following address-range is reserved for so called loopback addresses:
`127.0.0.0 – 127.255.255.255`

There is some more special ip-ranges, but for this project, you only need to remember those above.

back to contents

Masks

The network-mask, subnet-mask or in our project only called mask is there to decide which range of ip-adresses are part of the same subnet.
There are 2 different ways of writing the mask:

- "Dot-decimal notation": `255.255.255.0`
- "Class Inter-Domain Routing" or "CIDR": `/24`

The more usable ip-addresses you need in one subnet, the less subnets you will be able to create.
To help you understanding it, I found this table very helpful:

CIDR	Dot-decimal	Number of IP-addresses per subnet	Usable IP-addresses per subnet	Number of subnets
/32	255.255.255.255	1	0	256
/31	255.255.255.254	2	0	128
/30	255.255.255.252	4	2	64
/29	255.255.255.248	8	6	32
/28	255.255.255.240	16	14	16
/27	255.255.255.224	32	30	8
/26	255.255.255.192	64	62	4
/25	255.255.255.128	128	126	2
/24	255.255.255.0	256	254	1

The number of usable IP-addresses per subnet is lower than the total number of IP's because the first address is reserved as the network-address of the subnet and the last address is reserved as a broadcast-adress.
i.e. for mask `255.255.255.252`:
network: `190.3.2.252`
broadcast: `190.3.2.255`
usable IP's: `190.3.2.253`, `190.3.2.254`

back to contents

Switches

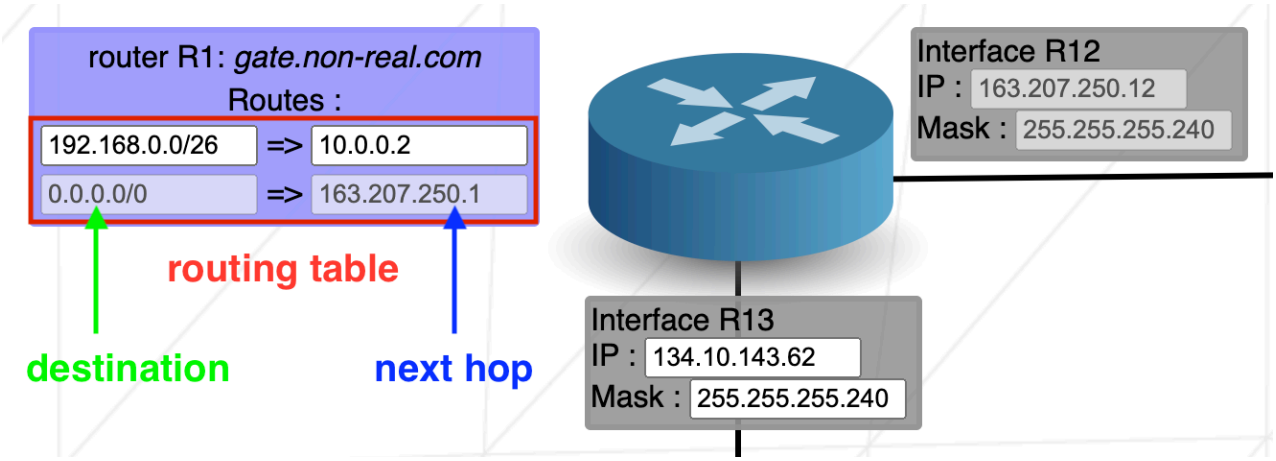
A switch will enable you to connect more than two devices to the same network.
Its only purpose is to distribute packages to its network.
To see a working example, you can take a look at Level 3.

Routers

As previously mentioned a router is an interface which enables communication between different networks.
A router has the ability to be part of multiple networks, in Netpractice this is visualized by the so called `Interface`.
If routers and switches are still magic to you, I suggest looking deeper into it yourself, as their basic understanding is crucial to succeed in this project.

back to contents

Routing Table



The routing table is there to store all the different paths to all the networks, the device is part of.
In Net_Practice the routing table consists of two elements, the **destination** and the **next hop**
The **destination** consists of the network-address that you want to send a package to, combined with the CIDR of that network: `190.3.2.252/30`. If you don't want to specify a destination, you can just set it to `default` or `0.0.0.0/0`.
The **next hop** is the address of the next router that you need to send the packages to in order to reach the destination-network.

back to contents

Network

And now to connect all of the above mentioned topics.
In order to have a functioning network, you now need to apply all of the parts talked about earlier.
If there should be a working connection in a network, the devices somehow need to be connected, either directly or with the help of routers which are part of both networks.

Now you may ask, how do I know if two devices are part of the same network?
For this you need to combine the IP-address and the mask of the devices in order to get the network-adress, that device is part of.
By combining I mean, doing a bit-by-bit-AND-opperation.
For that we first need to translate the IP and the mask to binary.
i.e.:
IP: `192.168.100.1` in binary: `11000000.10101000.1100100.00000001`
MASK: `255.255.255.0` in binary: `11111111.11111111.11111111.00000000`
Now you just combine the two bit by bit, if both bits are a `1` the corresponding bit of the network-address is `1`, in any other case the corresponding bit is `0`.

By doing that to the mentioned example, you should get the network-address of `11000000.10101000.1100100.00000000` in binary or `192.168.100.0` in dot-decimal.
If two devices share the same network-address, they are part of the same network and communication is ensured.

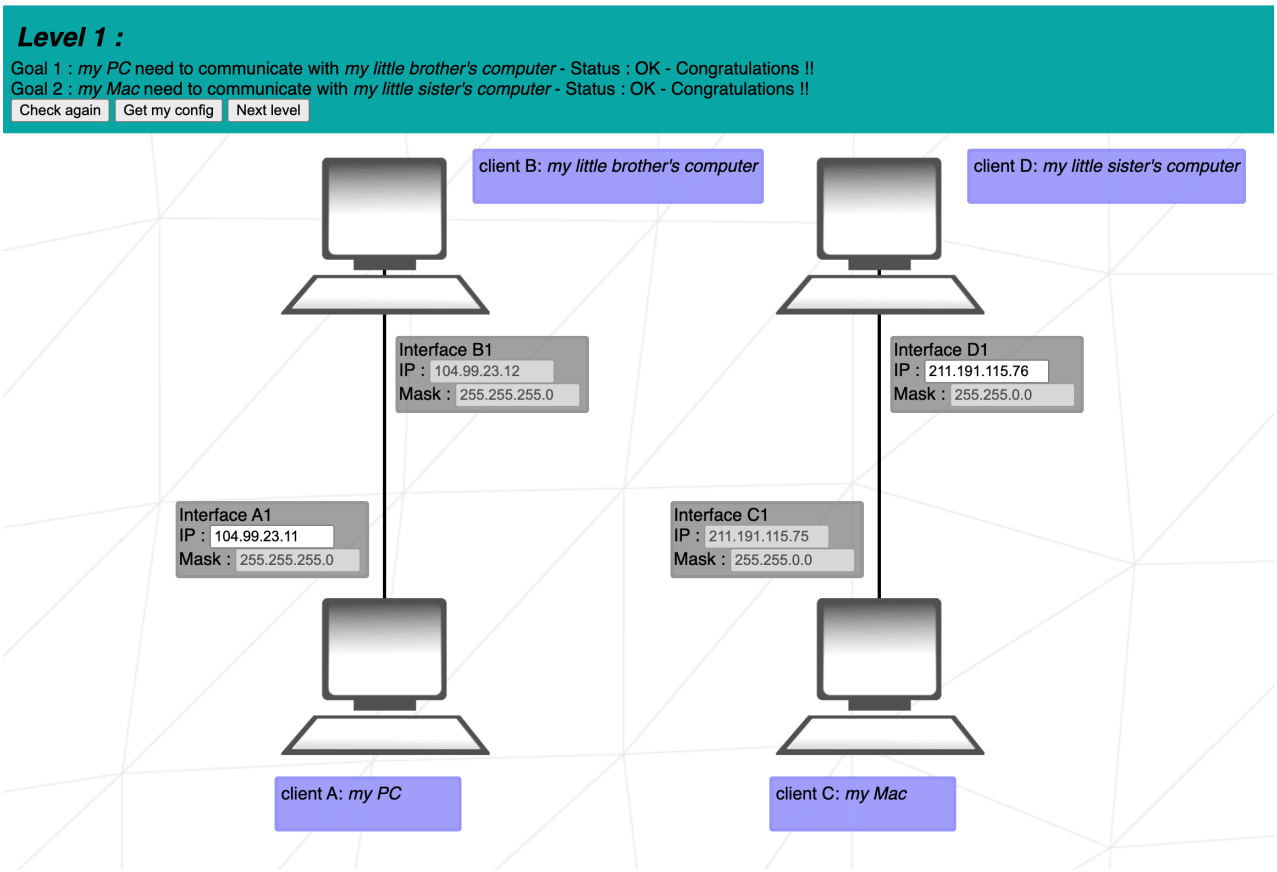
back to contents

Levels

Here are all the solutions and explanations for all 10 Levels.

Level 1

▼ show

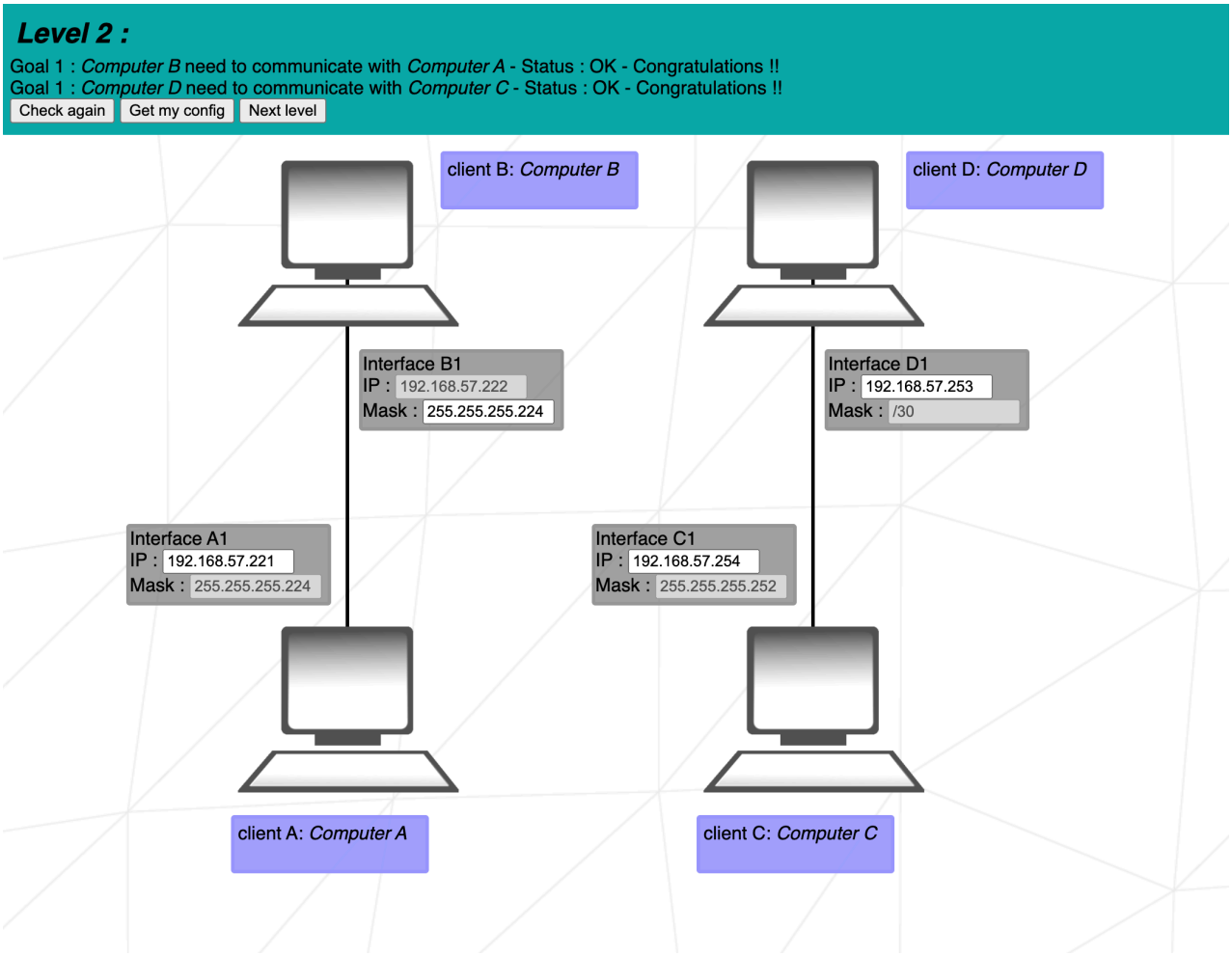


Here we have two separate networks, each consisting of two computers.
In order to make it work, the two computers need to be part of the same network.
Because the mask of A and B is `255.255.255.0` the possible IP-adresses of A1 are `104.99.23.1 - 104.99.23.254`.
For C and D the mask is `255.255.0.0`, so the usable IPs are `211.191.0.1 - 211.191.255.254`

back to contents

Level 2

README MIT license



Here we have two separate networks again, but this time we need to set the mask and IP correctly.
For the network of A and B, they need the same mask, `255.255.255.224`.
The available IP-addresses for A1 are `192.168.57.193 - 192.168.57.221`.

Network C and D already have the same mask. so they just need addresses that are part of the same subnet.
In the case of the mask being `/30` the subnet only consists of 2 available addresses per subnet. So be careful, to choose the correct ones. to make things easier, I suggest you either start with the lowest or highest subnet.
I choose the highest, so my available IP-range is `192.168.57.253 - 192.168.57.254`

back to contents

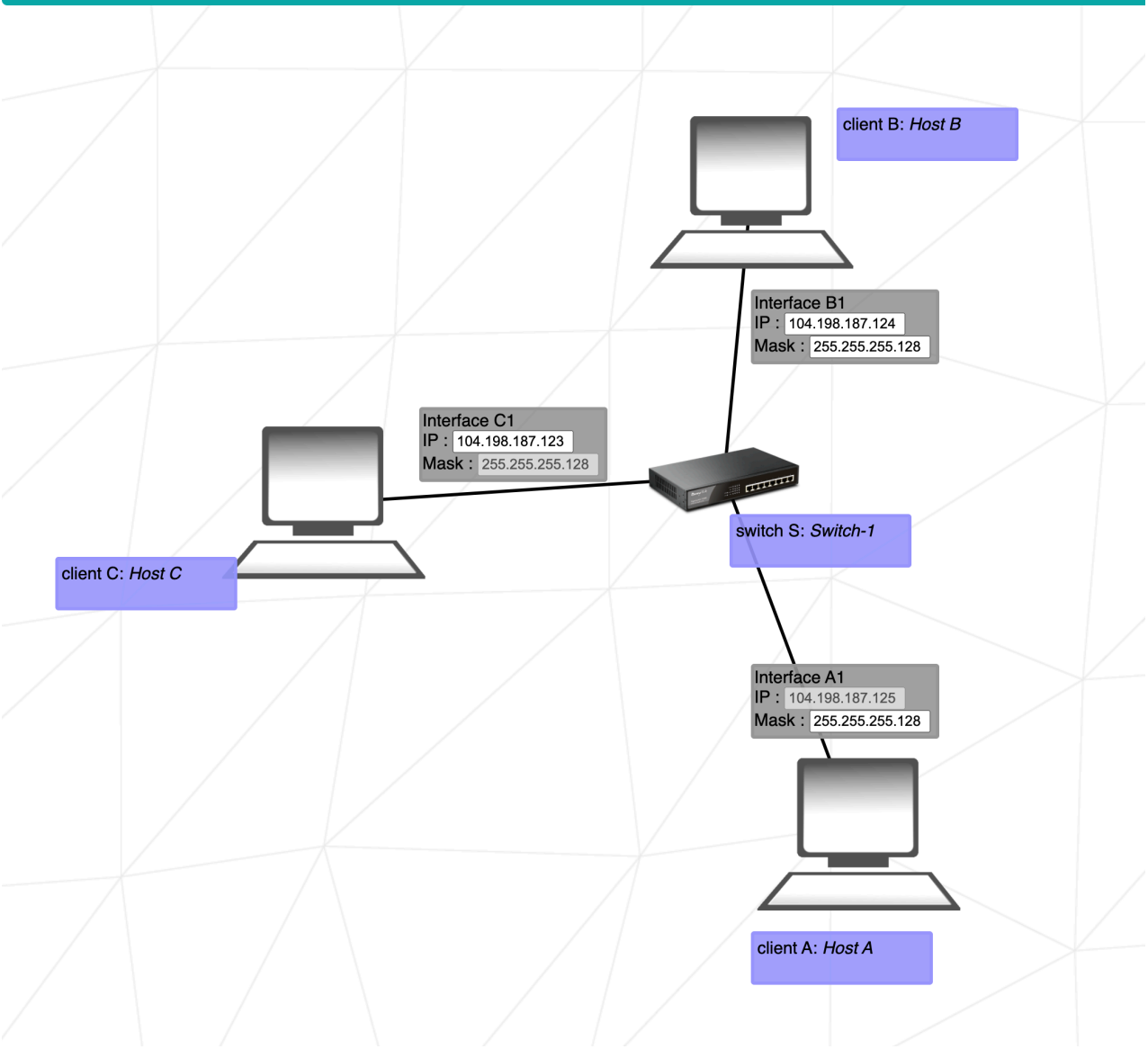
Level 3

▼ show

Level 3 :

Goal 1 : *Host A* need to communicate with *Host B* - Status : OK - Congratulations !!
Goal 2 : *Host A* need to communicate with *Host C* - Status : OK - Congratulations !!
Goal 3 : *Host B* need to communicate with *Host C* - Status : OK - Congratulations !!

Check again | Get my config | Next level



In this level we have our first encounter with a Switch.
Since we can't manipulate the mask of C, `255.255.255.128` will be the mask of the whole network.
A has a fix IP, so the whole networks range will be `104.198.187.1 - 104.198.187.126`.

back to contents

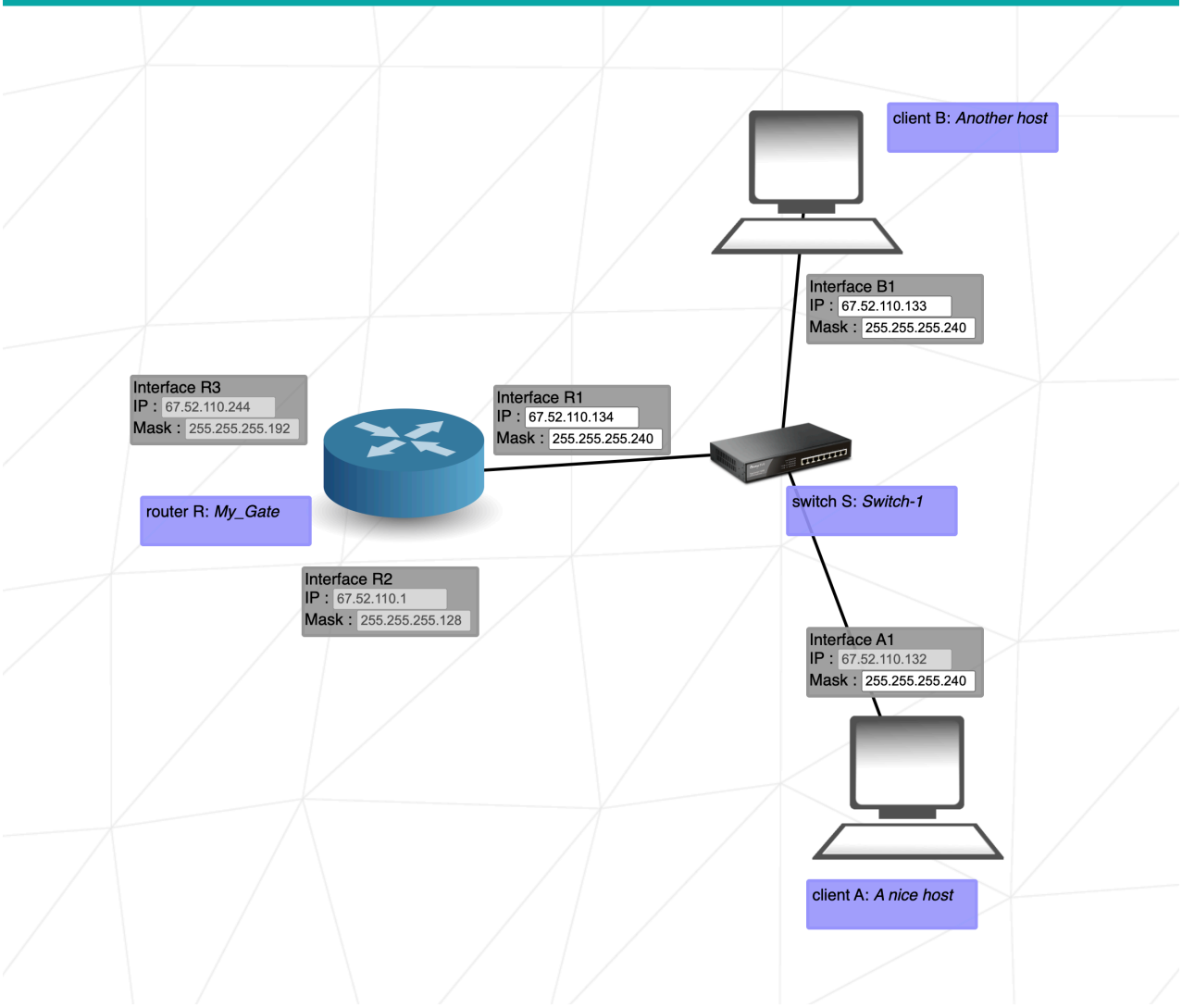
Level 4

▼ show

Level 4 :

Goal 1 : *A nice host* need to communicate with *Another host* - Status : OK - Congratulations !!
Goal 2 : *A nice host* need to communicate with *My_Gate* - Status : OK - Congratulations !!
Goal 3 : *Another host* need to communicate with *My_Gate* - Status : OK - Congratulations !!

Check again | Get my config | Next level



This is our first time running into a router, since the router does not connect to anything else, there is no routing table, that needs to be worked with.
A has a fixed IP, so this will define the IP of our network.
You can freely choose a fitting mask for the network, but the subnet has to have at least 3 usable IP-addresses,
so choosing `255.255.255.240` / `/28` will create a subnet of 14 usable addresses.
The fixed IP of A is `67.52.110.132`.
This results in an available IP-range of `67.52.110.129 - 67.52.110.142`.

back to contents

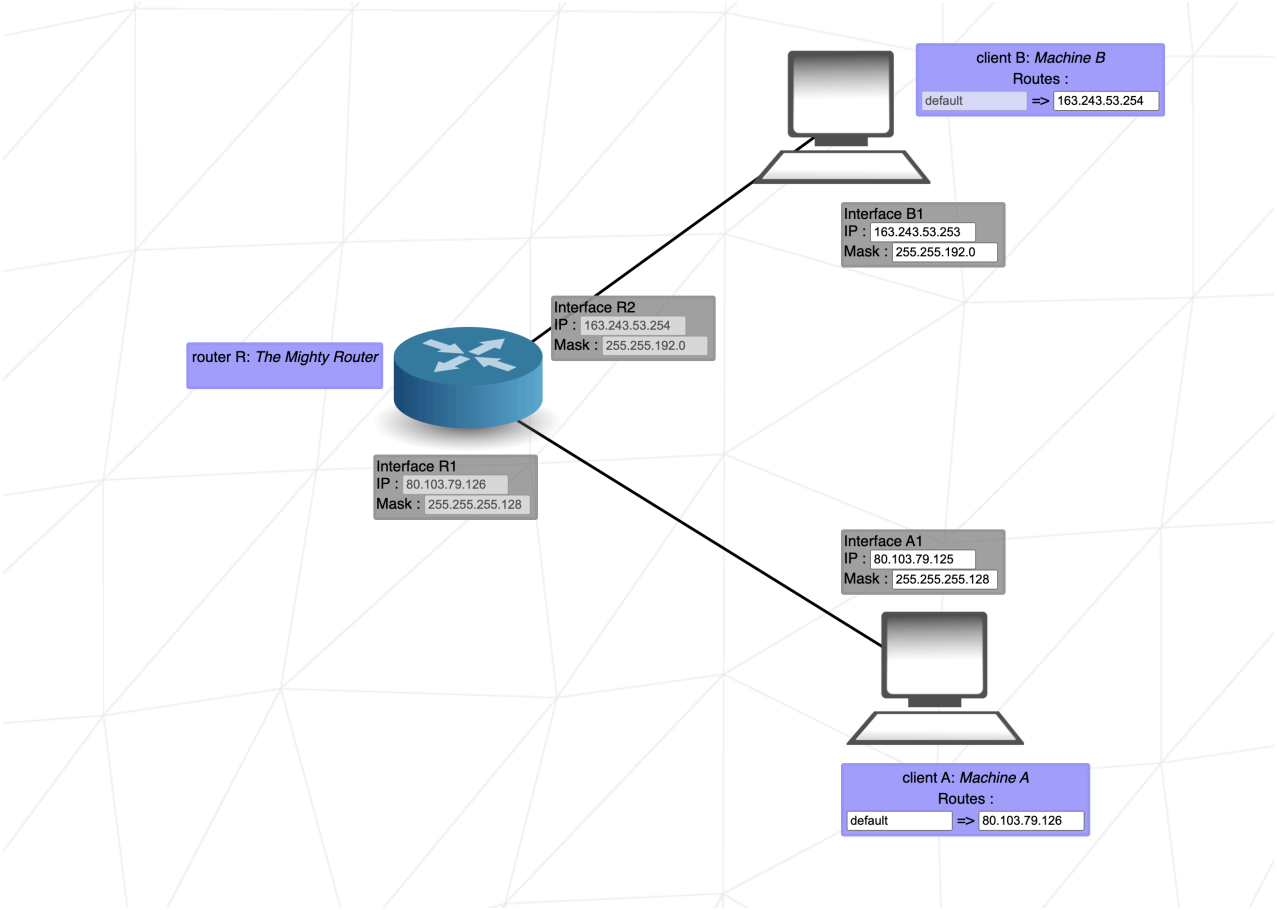
Level 5

▼ show

Level 5 :

Goal 1 : *Machine A* need to communicate with *The Mighty Router* - Status : OK - Congratulations !!
Goal 2 : *Machine B* need to communicate with *The Mighty Router* - Status : OK - Congratulations !!
Goal 3 : *Machine A* need to communicate with *Machine B* - Status : OK - Congratulations !!

Check again | Get my config | Next level



Here we first come by the mighty routing table.
R1 has a fixed IP of `80.103.79.126` and a fixed mask of `255.255.255.128`, which results in a mask of `255.255.255.128` and an IP-range of `80.103.79.1 – 80.103.79.125` for A1.

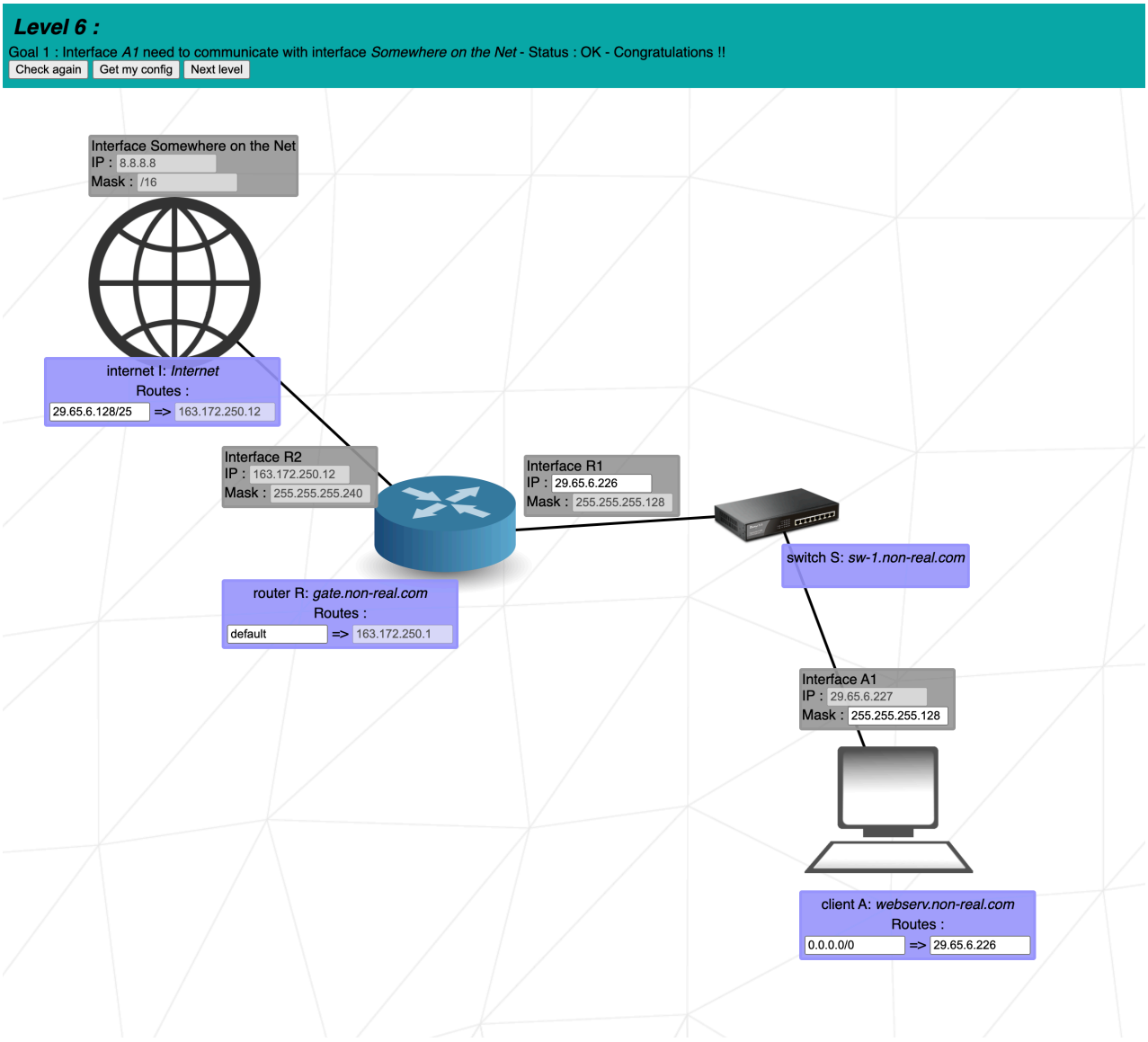
Now to conquer the routing table of A it is as easy as setting the **destination** as `default` or `0.0.0.0/0` and the destination has to be the IP of the directly connected router R1, `80.103.79.126`.

Same concept applies to connecting B to the R2.

back to contents

Level 6

▼ show



This level introduces us to the Internet.
We again start by looking at the fixed IP-addresses and masks.

IP of A1 is fixed and the mask of R1, which A1 is connected to through the switch, is fixed as well.
So we need to match those in order for them to be in the same network.
Mask for this network will be `255.255.255.128` which then will result in 2 subnets.
Combined with the fixed IP of `29.65.6.227` one subnet will be from `29.65.6.1` to `29.65.6.126` and the other
`29.65.6.129 – 29.65.6.254`.
In order for R1 and A1 to be in the same network, the possible address-range of R1 will be `29.65.6.129 – 29.65.6.254`.

Now set the routing table of A in order to reach R1.

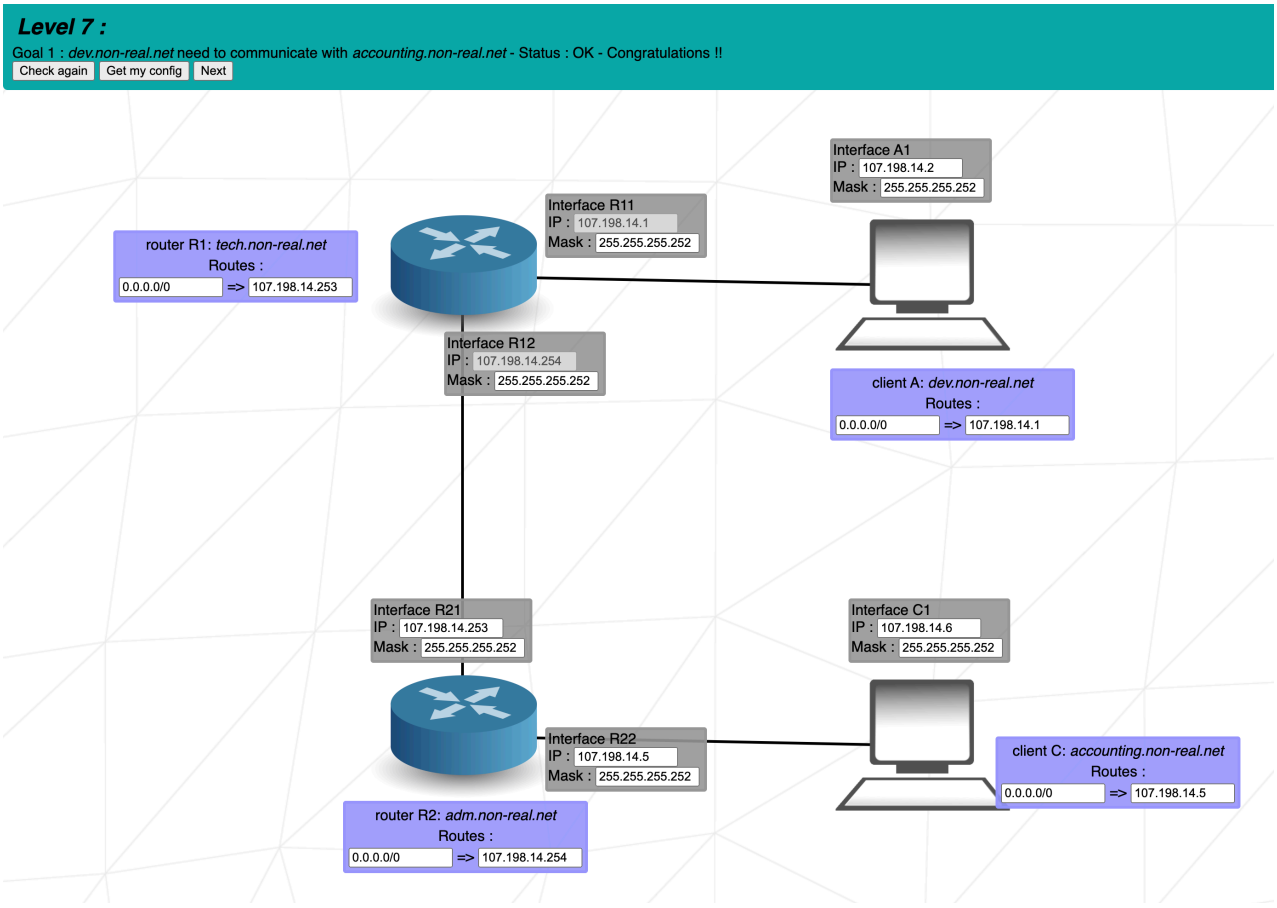
The **destination** of the routing table of the internet needs to be set to the network address of the R1-A1 network, which in this case is `29.65.6.128`, combined with the CIDR of the network, which is `/25`.
This results in a **next hop** of `29.65.6.128/25`.

Destination of the routing table of the router can be set to `default` or `0.0.0.0/0`.

back to contents

Level 7

▼ show



Level 7 is pretty straightforward and not too complicated, you just need to make sure that no networks overlap with each other.

The first goal will be to find an appropriate mask to use.
Since we will need 3 different subnets, by looking at the table you will be able to decide which mask will be required.
I did choose `255.255.255.252` or in CIDR `/30` as my mask because it provides me with subnets, each having 2 usable IP-addresses.
Now you just need to fill in all the correct IP's in order to create the 3 networks:

- A1 R11 (network range of `107.198.14.0 - 107.198.14.3`)
- R12 R21 (network range of `107.198.14.252 - 107.198.14.255`)
- R22 C1 (I did choose a network range of `107.198.14.4 - 107.198.14.7`)

back to contents

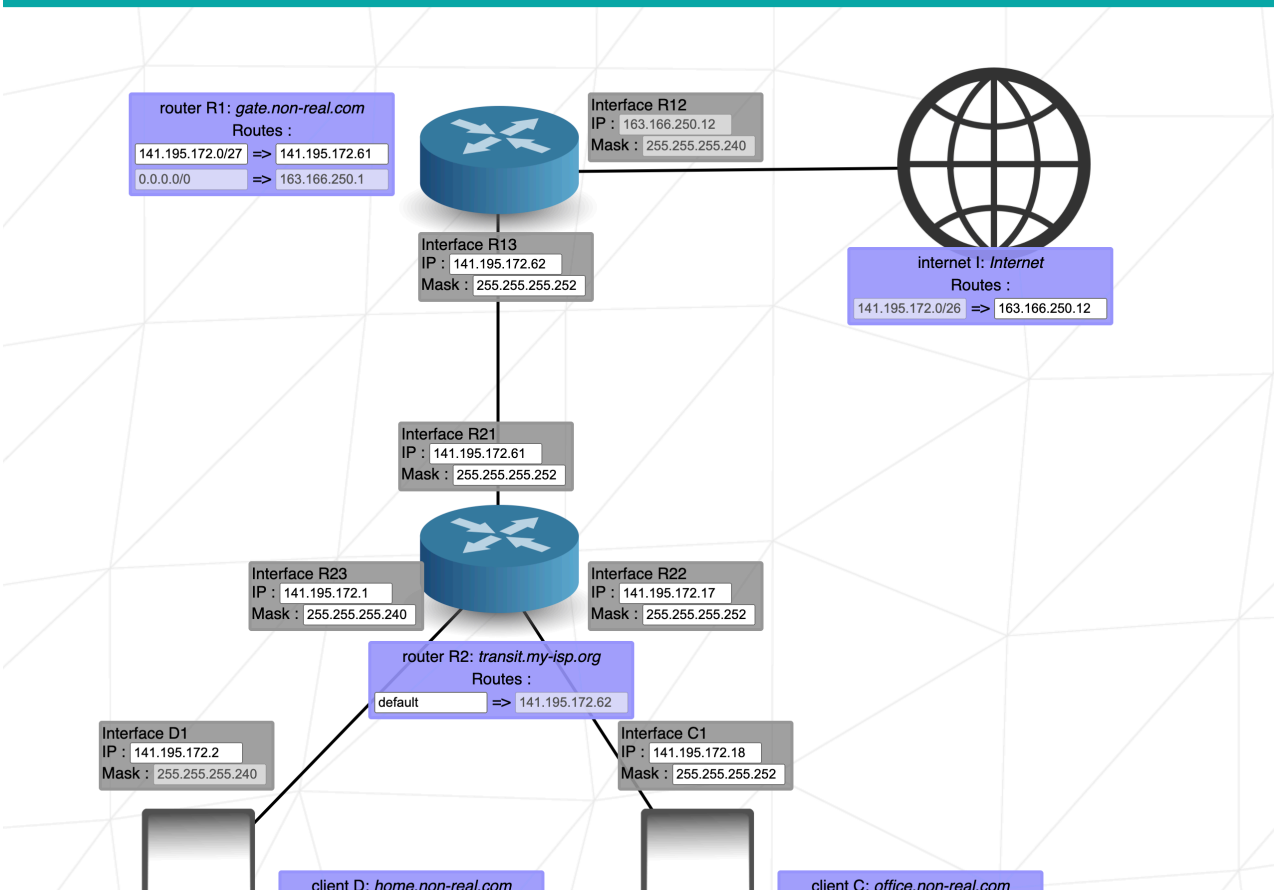
Level 8

▼ show

Level 8 :

Goal 1 : *office.non-real.com* need to communicate with *home.non-real.com* - Status : OK - Congratulations !!
Goal 2 : *office.non-real.com* need to communicate with *Internet* - Status : OK - Congratulations !!
Goal 3 : *home.non-real.com* need to communicate with *Internet* - Status : OK - Congratulations !!

Check again | Get my config | Next level



Level 8 now will be a little tricky because you really need to be aware of overlapping networks.

The first thing we can solve is the connection between R13 and R21.
The routing table of R2 gives you the fixed IP of R13 as `141.195.172.62` and as this network only consists of 2 needed IP-addresses we can set the mask of R13 and R21 to `255.255.255.252` in order to make things easier with overlapping IP-addresses.
Then this will result in `141.195.172.61` as the address of R21.

In R12 we see the fixed IP, which is `163.166.250.12`, so this needs to be put into the routing table of the Internet.

Now to set up the connection of D1 and R23.
First, set the mask of R23, since the mask of D1 is fixed to `255.255.255.240`.
Then put in 2 IP-addresses which are part of the same network intoR23 and D1, I decided to use the lowest subnet of the mask, since it is a not used IP-range yet, which is `141.195.172.1 - 141.195.172.14`.
So IP of R23 will be `141.195.172.1` and of D1 will be `141.195.172.2`.

For the network between R22 and C1 we are free to choose any mask we want. In order to make it as easy as possible I set it to `255.255.255.252`, because I only need 2 usable IP-addresses. After that I then choose a free range for my IP's.
In this case I did choose the next free one after the network of R23 and D1, which will be `141.195.172.17 - 141.195.172.18`.

Last thing to do is to set all the routing tables accordingly.
For C and D it's easy, both use `default` or `0.0.0.0/0` as a **destination** and the IP of the Interface of R2 they are directly connected to as **next hop**.

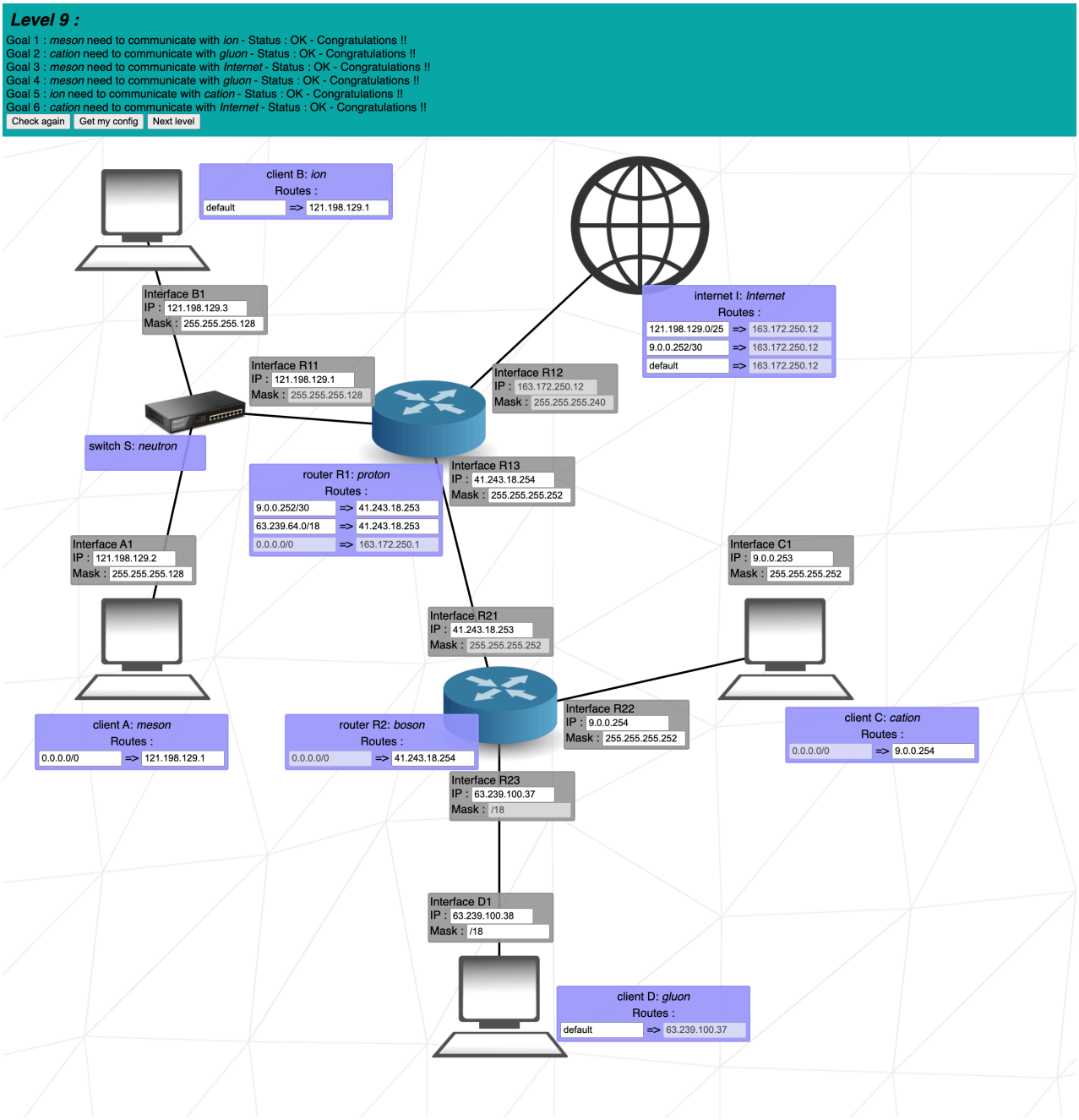
For the routing table of R1 it is a little harder.
Even though it would, for Net_Practice, work to just set the **destination** to `default` and get away with it just working, there is no way this should work as easily as this.
Because that would result in the routing table having 2 default destinations which lead to a different **next hop**, which if you think about doesn't make any sense, because the router would have no way of knowing which default to use.

To make it more sensible, the destination needs to be set to a value that leads to R2 and can go either way, to D1 and to C1.
Because we used the lowest two IP-ranges for the two target networks, that need to be reached, the **destination** will be `141.195.172.0/27` the IP adress is of the network, that we want to reach and the CIDR will just define that the first 27 bits of the IP have to be `141.195.172`, so it will be able to communicate with IP's ranging from `141.195.172.0` to `141.195.172.255`.

back to contents

Level 9

▼ show



In Level 9 we encounter another weakness of the project. The router is not doing any IP-translation when a computer reaches out to the internet.
This can be seen by trying to connect C to the internet.

First we set the ,mask of R22 and C1 to `255.255.255.252`, in order to block as little IP's as possible.
Now you have to decide on which IP-range to use for this network. I decided on the top-end, with the usable IP-addresses beeing `10.0.0.253` and `10.0.0.254`.
Now set the **next hop** in the routing table of C to the IP of R22.
Now change the routing table of R1, R2 and the Internet to make it work, as you learned before.

If you now hit the `check again` button, it will show you `Goal 6 : cation need to communicate with Internet - Status : KO - No reverse way, try again ...` for this demonstration we will just ignore all of the other goals.
Now you can see at the small red box at the bottom right corner:

```
******* Goal ID 6 ********
forward way : C -> I (163.172.250.1)
on C : packet accepted
on C : destination does not match any interface. pass through routing table
on C : route match 0.0.0.0/0
on C : send to gateway 10.0.0.254 through interface C1
on R2 : packet accepted
on R2 : destination does not match any interface. pass through routing table
on R2 : route match 0.0.0.0/0
on R2 : send to gateway 50.165.17.254 through interface R21
on R1 : packet accepted
on R1 : send to R12
on I : packet accepted
on I : destination reached
reverse way : I -> C (10.0.0.253)
on I : packet accepted
private subnets not routed over internet
```

From this we can see, the package from C reaches the Internet, but since the router only attached the private IP of C1 without translating it to a public IP, the Internet can't send any package back to a private IP.

All we have to do now to make it work, is to change every apperance of our `10.0.0.x` address to `9.0.0.x`.
Once you have done that you are greeted with
`Goal 6 : cation need to communicate with Internet – Status : OK – Congratulations !!`

So, for Net_Practice, a network can't have any IP which is part of a private Network, as soon as it will need connection to the Internet.

Now, all that is left is to solve the remaining 5 Goals.

3 Goals will be solved by creating the A-B-R11 network first, then connecting it to the Internet.
First, the mask of R11 is fixed, so set the same for B1 and A1.
Second, this network can not use private IP-addresses, so get rid of them.
Third, set the routing tables of A and B so they can reach R11.
Fourth, set the network address combined with the mask as a **destination** in the routing table of the internet.
In my case this was `106.198.154.0/25`.

The last two goals are depending on D to work.
As you can see in the routing table of D, the **next hop** is fixed, so put the same IP into R23.
The **destination** can be set to `default` or `0.0.0.0/0`.
Since the mask of that network is fixed as well, set it accordingly.

Now, if we didn't forget anything, all that should be left, is to fix the routing table of R1.
For this you need the network-address of the R23-D1 network.
You can find this with the same logic, as before by just extending the table above.
With this you will be able to find out that the mask `/18` divides our IP-range into 4 subnets:

- `63.239.0.0 – 63.239.63.255`
- `63.239.64.0 – 63.239.127.255`
- `63.239.128.0 – 63.239.191.255`
- `63.239.192.0 – 63.239.255.255`

Our fixed IP is part of `63.239.64.0 – 63.239.127.255`, so the network-address is `63.239.64.0`, combine this with our mask of `/18` and you have the missing **destination** of our routing table, `63.239.64.0/18`.

back to contents

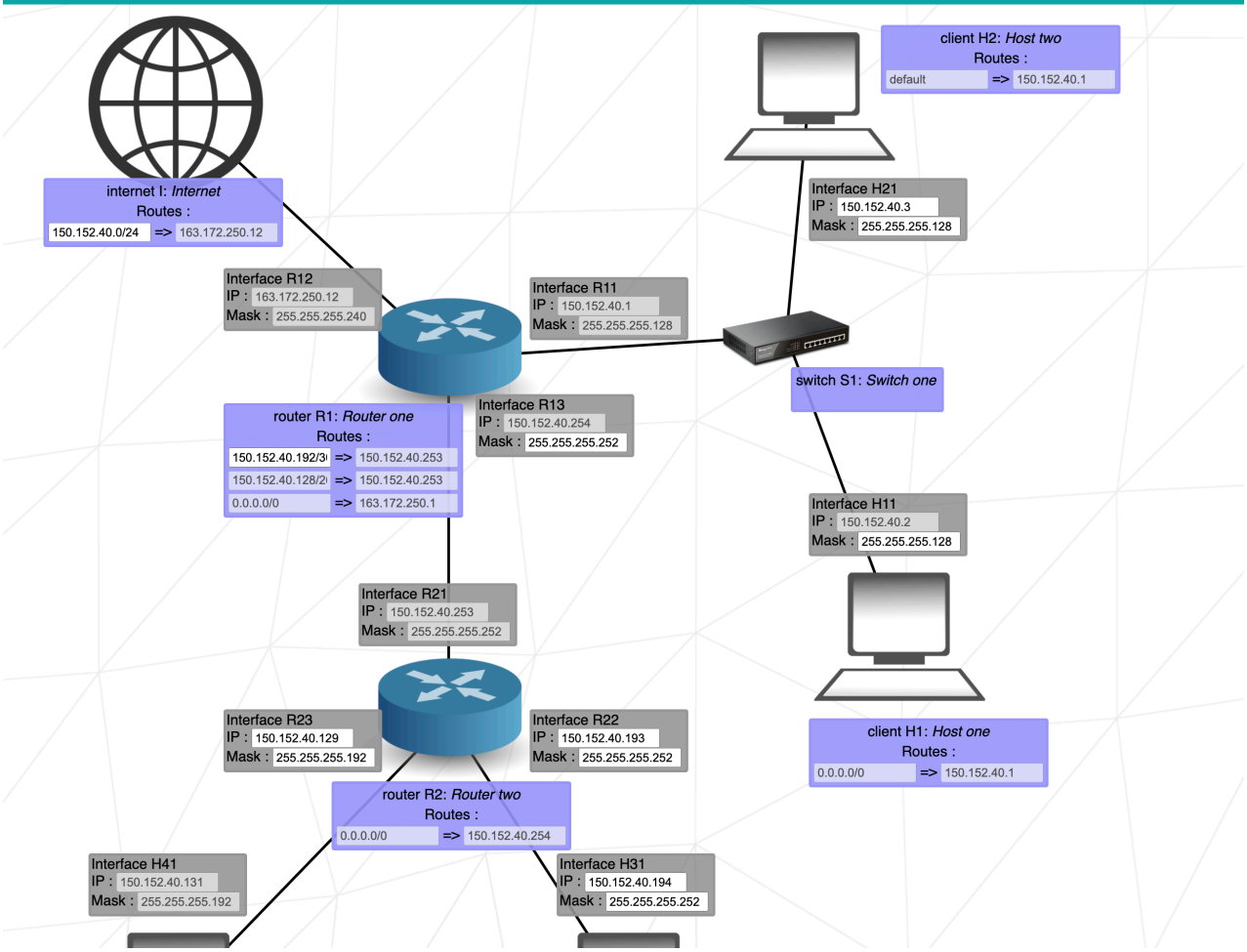
Level 10

▼ show

Level 10 :

Goal 1 : *Host one* need to communicate with *Host two* - Status : OK - Congratulations !!
Goal 2 : *Host three* need to communicate with *Host four* - Status : OK - Congratulations !!
Goal 3 : *Host one* need to communicate with *Internet* - Status : OK - Congratulations !!
Goal 4 : *Host one* need to communicate with *Host four* - Status : OK - Congratulations !!
Goal 5 : *Host two* need to communicate with *Host three* - Status : OK - Congratulations !!
Goal 6 : *Host three* need to communicate with *Internet* - Status : OK - Congratulations !!
Goal 7 : *Host four* need to communicate with *Internet* - Status : OK - Congratulations !!

Check again | Get my config | Complete !



The easiest part will be the connection of H2 and H1 in their network.
The mask of this network is fixed by R11, set the masks of H21 and H11 accordingly.
Set the IP of H21 to match the rest of the network, it can be in the range of `158.103.36.3 – 158.103.36.126`.

Now we are setting up the H41-R23 network and its connection to the Internet.
The mask is fixed, so set it accordingly.
The IP of R23 is fixed by the routing table of H4, so set it to the correct one.
Now go along the route connecting it to the Internet, you will notice the mask of R21 is fixed so set the mask of R13 accordingly.
The routing table of R1 already has the default set, as a connection to the internet.
So now the package should be able to reach the Internet and only thing to fix is the routing table of the Internet.
For this to be correct we only need to change the CIDR of the **destination**, so it is able to communicate with all `158.103.36.x` addresses.
This is done by changing the CIDR to `/24`, so it can reach `158.103.36.0 – 158.103.36.255`.

The hardest part of this level will now be the setup of the R22-H31 network and the correct setup of the routing table of R1.
First, set the mask of R22 and H31 to `255.255.255.252` since we only need 2 usable IP-addresses and this creates the least problems with overlapping ip-ranges.
Second, you need an IP-range that is still free. For this, we will take a look at all of the other networks.

- `150.152.40.0 – 150.152.40.127` is taken by R11-H21-H11
- `150.152.40.128 – 150.152.40.191` is taken by R23-H41
- `150.152.40.252 – 150.152.40.255` is taken by R13-R21

So my decision was to take the next free part after `150.152.40.128 – 150.152.40.191`, which with the `/30` mask will be `150.152.40.192 – 150.152.40.195`.
Set the addresses accordingly to the usable ones of the range, you decided on.

Lastly, for the routing table of R1 we need the network-address of the R22-H31 network, which in my case is `150.152.40.192/` and it's mask, which is `/30`, set the **destination** accordingly.

back to contents

Contributors 4

 **tblaase** Tim Blaase

 **ayogun** Ali Ogun

 **ilp-sys** ilp-sys

 **raweber42** raweber